Int. J. Production Economics 268 (2024) 109102

Contents lists available at ScienceDirect

International Journal of Production Economics

journal homepage: www.elsevier.com/locate/ijpe

A deep reinforcement learning based algorithm for a distributed precast concrete production scheduling

Yu Du [a], Jun-qing Li [b,c,*]

[a] School of Information Science and Engineering, University of Jinan, Jinan 250022, Shandong, China
[b] School of Mathematics, Yunnan Normal University, Kunming, 650500, Yunnan, China
[c] School of Information Engineering, HengXing University, Qingdao 266100, Shandong, China

ARTICLE INFO

Keywords:
Reinforcement learning
Precast concrete production
Distributed flexible job shop scheduling problem
Group scheduling
Time-of-use electricity price

ABSTRACT

The environmental-friendly production demands higher manufacturing efficiency and lower energy cost; therefore, time-of-use electricity price constraint and distributed production have attracted more attention. For concrete precast architecture construction, the tardiness penalty and warehouse cost cannot be ignored, which should be optimized for lower cost. In this study, the concrete precast process is investigated as the group scheduling of a distributed flexible job shop problem. Each concrete precast is managed in a group, the setup time between different groups is considered. Two objectives, total weighted earliness and tardiness and total time-of-use electricity cost are minimized, simultaneously. To solve the integrated problem, three coordinated double deep Q-networks (DQN) are applied, which are organized as a learn-to-improve reinforcement learning approach. For distributed scheduling problem, operators in a single factory or in multiple factories differs in solution improvement; so selection DQN is designed to decide the type of the operators according to scheduling circumstances. Other two DQNs, i.e., local DQN and global DQN, are to select the optimization operators in one factory or in multiple factories, respectively. Furthermore, two solution refinement strategies are designed to decrease the objectives after reinforcement learning component. Numerical experiment and statistical analysis suggest that the proposed deep reinforcement learning based algorithm has superiority in solving the considered problem.

1. Introduction

Traditional concrete buildings consume large energy, so pre-fabricated buildings were developed to reduce energy consumption, which has improved environmental-friendly manufacturing (Ma et al., 2018; Qi et al., 2023). The precast concrete (PC) manufacturing has major effect on the efficiency of pre-fabricated construction; therefore, the scheduling optimization of PC manufacturing and its efficiency has crucial impact on pre-fabricated construction.

However, the PC manufacturing is mainly modeled as flow-shop scheduling problem (FSP) that easily be affected by critical operations, resulting in the difficulties in improving production efficiency. With the development of multi-functional machines, machines in PC manufacturing factory can perform multiple operations; therefore, this study treated the PC production process as the flexible job shop scheduling problem (FJSP) in each factory. Furthermore, single factory cannot satisfy the assembly and transportation demands of PC manufacturing; as a consequence, this study considers the PC production process as distributed FJSP (DFJSP-PC). In DFJSP-PC, Each PC is seen as a job in the DFJSP environment. In the PC manufacturing, an order includes multiple types of PC products, where each type of PC contains several jobs that have similar production process and technique. Each type of job can be seen as a family in group scheduling constraint. In addition, time-of-use electricity price (TOUEP) constraint is also considered in calculating the total electricity cost (TEC) for better balancing energy distribution and satisfying environmental-friendly urge.

Most DFJSP studies selected makespan (maximum completion time) as a minimized objective; however, for prefabricated building construction, PC is hard to deposited for its huge volume, leading to the high cost of warehousing. Meanwhile, the tardiness of PC apparently affects the whole construct process. Therefore, this study selects the total weighted earliness and tardiness (TWET) as one of the objectives. The TWET not only considers the tardiness penalty of each job, but also guarantees the scheduling feasibility.

As a complex type in combinatorial optimization problems (COP), DFJSP is usually solved by evolutionary algorithms (EA) and mathematical programming. Most published EA for solving COP are manually engineered (Ma et al., 2022), where the data characteristics and

* Corresponding author at: School of Mathematics, Yunnan Normal University, Kunming, 650500, Yunnan, China.
E-mail addresses: dyscupse@163.com (Y. Du), lijunqing@lcu-cs.com (J.-q. Li).

https://doi.org/10.1016/j.ijpe.2023.109102
Received 23 July 2022; Received in revised form 8 November 2023; Accepted 13 November 2023
Available online 21 November 2023

scheduling features should be considered to improve the performance in solving COP. Mathematical programming can obtain global optima for small-sized COP; however, for medium- and large-sized COP, it cannot find feasible solution in satisfying time. Furthermore, the modeling of mathematical programming for complex constrained COP is difficult.

Compared with mathematical programming and heuristic models, reinforcement learning (RL) approaches can better balance the exploration and exploitation abilities in iteration search, and the state features in RL can be the natural indicators with self-adaptive ability. For the discrete combinatorial processes of complex system, RL has advantages in solving sequential decision-making problems (Rolf et al., 2022; Jahani et al., 2023). With appropriate design, the size of RL model is fixed in any scale of problem, and the trained RL model can be directly used in real scheduling optimization. Deep reinforcement learning (DRL) combines the generalization of deep learning and the policy selection of RL. In 2015, Mnih et al. (2015) proposed a DQN model to solve Atari games, providing an efficient approach and feasibility in solving complex problems.

After DQN training stage, the optimal policy trained by the proposed algorithm can select appropriate action in varieties of scheduling status, contributing faster convergence in iteration and better optimization objectives. Practically, the final solutions obtained by the optimal policy can be directly applied in the corresponding scheduling circumstances. Specifically, the real PC manufacturing can refer the distribution of different groups of jobs, the product sequence of all operations, and the machine assignment of each operation obtained by the given solution.

This study constructs a DQN-based evolutionary algorithm with solution refinement strategies (DQN-EA-SR) to solve DFJSP-PC, the main contributions of this study are listed as follows.

- The DFJSP-PC is modeled with TOUEP constraint, where two objectives, i.e., TWET and TEC, are optimized simultaneously;
- Group scheduling and the setup times between different group jobs are considered;
- Three functional DQNs are integrated to select search strategies in various scheduling circumstances;
- In the DQN-EA-SR, three double DQNs are integrated in RL component, where one DQN is to decide the action type, the other two DQNs are with action execution;
- Two solution refinement strategies are designed to further optimize the two objectives after RL component.

The remainders of this paper are organized as follows. Section 2 presents a review of the related constrained scheduling studies, DFJSP studies, and RL- and DRL-based researches for production scheduling. The problem description and objective definitions of the considered DFJSP-PC are provided in Section 3. The details of DRL-EA-SR are given in Section 4. The experimental analyses are discussed in Section 5. Finally, conclusions of this study are drawn in Section 6.

2. Literature review

2.1. Constrained scheduling

As for PC scheduling, Anvari et al. proposed a multi-objective GA-based searching technique to solve a unified manufacturing, transportation, and assembly FJSP (Anvari et al., 2016). Liu et al. designed a heuristic algorithm with eight sets of conjoint priority rules for ready-mixed concrete plant scheduling with multiple mixers (Liu et al., 2017). Ma et al. used an FSP to optimize the multiple production lines manufacturing of reinforced precast concrete components (Ma et al., 2018). Kim et al. provided a dynamic model for PC construction under due date constraint (Kim et al., 2020). Xiong et al. considered an integrated distributed concrete precast flow shop system including daily working, daily non-working, and allowable overtime working time, providing a model based on realized application (Xiong et al., 2021). Jiang and Wu treated PC production process as flow-shop with setup times, optimizing total tardiness and earliness (Jiang and Wu, 2021). Li et al. modified the precast components manufacturing process into distributed hybrid FSP, where the makespan and total energy consumption were optimized simultaneously (Li et al., 2022a). Specially, aiming to PC production environment, Kim et al. applied a DQN model to optimize total tardiness, where four dispatching rules were developed as actions (Kim et al., 2022).

As for group scheduling, Xu et al. investigated a single-machine group scheduling problem with non-periodical maintenance and deteriorating effects, where the related properties, two batch-based heuristics, and an iterated greedy algorithm were proposed (Xu et al., 2019). Pan et al. considered a novel distributed flowshop group scheduling environment, and proposed an effective cooperative co-evolutionary algorithm (Pan et al., 2020). Zhao et al. established a mixed integer linear programming (MILP) model and a memetic algorithm for an industrial group scheduling problem, where the number of late jobs and total setup time were optimized simultaneously (Zhao et al., 2020). Cheng et al. developed two new benchmark algorithms for no-wait flowshop group scheduling problem with sequence-dependent setup times (Cheng et al., 2021). Yuraszeck et al. combined a decomposition approach with a constraint programming to solve a fixed group shop scheduling problem, where total flow time was minimized (Yuraszeck et al., 2022). Hosseinzadeh et al. proposed two MILP models and two meta-heuristics algorithms to solve group scheduling problem with sequence-dependent setup times (Hosseinzadeh et al., 2022).

As for TOUEP constrained scheduling, Wang et al. solved a two-machine permutation FSP by minimizing TEC under TOUEP constraints (Wang et al., 2018). Najafzad et al. presented a bi-objective optimization model for multi-skill project scheduling problem under TOUEP tariffs, in which the project total cost and makespan were minimized simultaneously (Najafzad et al., 2019). Jiang and Wang solved FJSP under TOUEP constraints by a multi-objective optimization algorithm based on decomposition, where a MILP model is proposed (Jiang and Wang, 2020). Saberi-Aliabad et al. investigated an unrelated parallel-machine environment under TOUEP tariffs, aiming to minimize the cost of consuming energy (Saberi-Aliabad et al., 2020). Ho et al. considered TOUEP tariffs in flow-shop scheduling problem, in which electricity cost and makespan were minimized (Ho et al., 2021). Cao et al. applied TOUEP constraints to iron-steel plant with self-generation equipment environment (Cao et al., 2021).

From the above scheduling studies with PC production, group scheduling, and TOUEP constraints, few of them considered complex scheduling problem, such as constrained FJSP and DFJSP.

2.2. Studies on DFJSP

In 2010, De and Pezzella proposed an improved GA for DFJSP, which is the milestone in solving DFJSP by efficient approach (De Giovanni and Pezzella, 2010). Then, numerous algorithms and strategies have been developed and designed to solve DFJSP. Chang and Liu constructed a hybrid GA to solve DFJSP, providing a novel encoding mechanism to solve invalid job assignments encoding (Chang and Liu, 2017). Wu et al. designed an improved differential evolution algorithm to solve DFJSP with assembly process (Wu et al., 2019). In recent years, Meng et al. gave four different MILP models and a constraint programming model for DFJSP (Meng et al., 2020). Luo et al. established an effective memetic algorithm for solving DFJSP, where operations from a job can be performed at different factories (Luo et al., 2020). Du et al. designed a hybrid algorithm of EDA and VNS to solve a complex constrained DFJSP, where makespan and TEC were minimized (Du et al., 2021), and Li et al. considered the similar complex FJSP with crane transportation (Li et al., 2022b). Wang et al. adopted an evolutionary game-based solver method to solve the DFJSP under industrial internet-of-things background (Wang et al., 2021). Xu et al. developed a hybrid algorithm of GA and TS to solve the DFJSP concerning operation outsourcing and carbon emission (Xu et al., 2021). Sang

and Tan combined NSGA-III and local search methods to solve the many-objective DFJSP, where makespan, TEC, machine load, delay time, and processing quality were optimized simultaneously (Sang and Tan, 2022). Li et al. proposed a bi-population balancing multi-objective EA for DFJSP in steel manufacturing system, where maximum fuzzy completion time and the energy consumption are minimized (Li et al., 2023).

Although most of the studies on DFJSP that solved by EAs obtained satisfying outputs, the intelligence of machine learning and reinforcement learning cannot be ignored.

2.3. RL for production scheduling

The researches in RL for production scheduling can be mainly categorized into two types, i.e., *end-to-end* and *learning-to-improve* approaches. The function of the RL in both end-to-end and learning-to-improve is as a strategy selector. The *end-to-end* approach completes the scheduling solution by implementing each encoding element from starting to ending, where a long-time iteration optimization is not needed. The actions in RL of the end-to-end approach are mainly dispatching rules or integrated dispatching rules, which are the essential component in designing RL-based algorithms. The *learning-to-improve* approach refines the initialized solutions with searching strategies and heuristics, so it consumes more time compared with end-to-end approach but the obtained solutions generally have high quality for iteration improvement.

For studies in the *end-to-end* approach, Li et al. applied a hybrid Q-network for dynamic FJSP, where the makespan and total energy consumption were optimized (Li et al., 2022c). For semiconductor packaging facilities, Park and Park designed a DQN model for the FJSP with sequence-dependent setups of job and operation types (Park and Park, 2021). Pan et al. proposed a DRL-based optimization algorithm for permutation flow-shop scheduling problem, where the network was trained by actor–critic method (Pan et al., 2021). Luo et al. built a DQN (Luo, 2020) and a two-hierarchy DQN (Luo et al., 2021a) for dynamic FJSP, where the tardiness and machine utilized were optimized simultaneously. For dynamic partial-no-wait multiobjective FJSP, Luo et al. designed a DQN with hierarchical multi-agent proximal policy optimization (PPO) (Luo et al., 2021b). Liu et al. constructed a hierarchical and distributed architecture to solve dynamic FJSP, where routing and sequencing agents were included (Liu et al., 2022). Du et al. designed a novel DQN model to solve the FJSP with eight different crane transportation modes, machine setup, and machine idle constraints (Du et al., 2022b).

For researches in the *learning-to-improve* approach, Chen et al. built a RL-based self-learning GA for solving FJSP, where SARSA and Q-learning were used at learning stage (Chen et al., 2020). Lin et al. presented a Q-learning based hyper-heuristic algorithm to solve semiconductor final testing problem, where the makespan was optimized (Lin et al., 2022). Wang and Wang designed a cooperative memetic algorithm with a RL-based policy agent for energy-aware distributed hybrid FSP (Wang and Wang, 2021). Du et al. developed an DQN-based optimization algorithm for an integrated FJSP, where the makespan and total energy consumption were optimized simultaneously (Du et al., 2022a).

The before-mentioned papers mainly focus on solving basic scheduling problems by using end-to-end and learning-to-improve approaches; however, more realistic manufacturing problem should be analyzed. The DFJSP-PC is a distributed scheduling problem where the job distribution greatly impacts the solution. Unlike end-to-end approaches, learning-to-improve methods facilitate iterative optimization of job distribution, even when the initial solution is suboptimal. Furthermore, the considered DFJSP-PC is a complex problem involving time-of-use electricity price, setup time, job families, precast concrete production constrains. End-to-end approaches require a deep understanding of the problem to design effective dispatching rules for high-quality solutions. In contrast, learning-to-improve approaches offer more flexible and diverse heuristics for iterative solution adjustment and optimization, which simplified the design of optimization heuristics for learning-to-improve approaches. Therefore, this study applied integrated double DQNs (DDQNs) to solve the DFJSP-PC.

3. Problem description

In the considered DFJSP-PC, Λ families ($\Lambda \geq 2$) of jobs are distributed to F factories ($F \geq 2$). All jobs from one family should be assigned to the same factory. Each factory has K machines, where a FJSP with setup constraint environment is set. Each job has six operations: (1) mold assembling, (2) reinforcement and embedded parts setting, (3) concrete casting, (4) concrete curing, (5) mold stripping, and (6) production finishing and repairing. The six operations have to be performed in sequence. For each operation, a set of available machines of assigned factory can be selected. The last machine of each factory is to complete concrete curing, where the machine can perform more than one jobs simultaneously. For other operations, the assigned machine can only perform one job at a time. The setup stage is considered when two consecutive operations from jobs of different families are performed on one machine. The energy contains machine processing and setup stages. In concrete curing, setup time is ignored in processing time.

3.1. Notations

Indices and parameters:

- Λ: number of families, in this study, $\Lambda \geq 2$;
- λ: index of families, $\lambda \in \{1, \dots, \Lambda\}$;
- F: number of factories, in this study, $F \geq 2$;
- f: index of factories, $f \in \{1, \dots, F\}$;
- I: number of jobs in all families;
- i: index of jobs, $i \in \{1, \dots, I\}$;
- λ_i: family that job i from;
- OP_i: number of operations in job i;
- j: index of operations, $j \in \{1, \dots, OP_i\}$;
- K: number of machines in each factory;
- k: index of machines, $k \in \{1, \dots, K\}$, the Kth machine in each factory is only for concrete curing;
- P: number of all operations, $P = \sum_{i=1}^{I} OP_i$;
- p: the performed sequence index of the corresponding operation in all operations, $p \in \{1, \dots, P\}$;
- M: number of electricity price intervals;
- m: index of electricity price intervals, $m \in \{1, \dots, M\}$;
- d_i: due date of job i;
- E_i, T_i: earliness and tardiness of job i;
- α_i, β_i: unit earliness and tardiness penalty cost of job i;
- $O_{i,j}$: the jth operation of job i;
- $p_{i,j,f,k}$: processing time of $O_{i,j}$ assigned on the kth machine in factory f;
- $s_{f,k,\lambda_1,\lambda_2}$: setup time between jobs from families λ_1 and λ_2 on the kth machine in factory f;
- $PS_{i,j}$, $PC_{i,j}$: starting and completion time of $O_{i,j}$;
- $MS_{f,k,p}$, $MC_{f,k,p}$: starting and completion time of the pth operation assigned on the kth machine of factory f, respectively;
- $SS_{f,k,p}$, $SC_{f,k,p}$: starting and completion times of setup stages, where the corresponding operation is the pth performed operation that assigned on the kth machine in factory f, respectively;
- TEC_p, TEC_s: total electricity cost of machine processing and setup stages, respectively;
- EP_m: electricity price for the mth electricity price interval;
- $tp_{f,k,p,m}$, $sp_{f,k,p,m}$: machine processing, and setup in the mth electricity price interval of the pth performed operation on the kth machine in factory f, respectively;

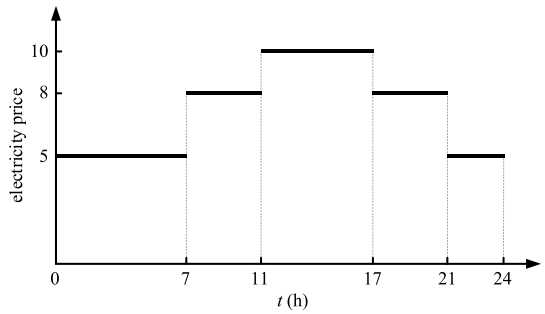


Fig. 1. TOUEP and working time settings in a day.

- $X_{i,j,f,k,p}$: a binary variable, equals 1 when $O_{i,j}$ is assigned on the pth performed sequence of machine k of factory f; otherwise, 0.

3.2. Assumptions

- Each operation can only be performed by one machine;
- All machines cannot be shut down before the completion of all assigned operations;
- Each factory has to perform at least one family in scheduling assignment;
- The considered scheduling starts at time 0;
- After an operation is performed, if the corresponding machine has its successor operation from other family, setup stage is executed after the former operation;
- In concrete curing, only the last machine in factory can perform the jobs; and other machines can perform all processes except concrete curing;
- No disruption events are included.

3.3. TOUEP setting

In this study, the TOUEP constraint is considered, indicating that the electricity consumption is decided by not only performing time length but working time position in a whole day.

The TOUEP setting in a whole day is illustrated in Fig. 1, which is similar as Luo et al. (2013). In Fig. 1, the high electricity price is in 11:00 am–17:00 pm, the medium electricity price is in 11:00 am–11:00 am and 17:00 pm–21:00 pm, other time is in low electricity price. The TOUEP setting is the same every day.

3.4. Optimization objectives

Two objectives, i.e., TWET and TEC, are minimized simultaneously. The value of the TWET can be obtained by (1).

$$TWET = \sum_{i=1}^{I} \left(\alpha_i E_i + \beta_i T_i\right) \tag{1}$$

$$E_i = \max\{d_i - PC_{i,6}, 0\}, \forall i \in I \tag{2}$$

$$T_i = \max\{PC_{i,6} - d_i, 0\}, \forall i \in I, \tag{3}$$

where d_i is the due date of job i.

The value of the TEC is calculated by (4), where the TEC is the total electricity cost of machine processing and setup.

$$TEC = \sum_{f=1}^{F}\sum_{k=1}^{K}\sum_{p=1}^{P}\sum_{m=1}^{M}\left(tp_{f,k,p,m} * EP_m\right) + \sum_{f=1}^{F}\sum_{k=1}^{K}\sum_{p=1}^{P}\sum_{m=1}^{M}\left(sp_{f,k,p,m} * EP_m\right); \tag{4}$$

3.5. Problem-specific definition and properties

According to the problem description, two problem-specific properties are proposed, providing knowledge-based foundation of actions and refinement strategies.

Property 1. *If all operations are performed as early as possible with Eqs. (5)–(7), the earliness and tardiness of all jobs cannot be decreased unless the scheduling sequence of operations is changed.*

$$PS_{i,j} = \max\{SC_{f,k,p}, PC_{i,j-1}\}, \quad \forall i, j \in \{2, \ldots, OP_i\}, \\ f, k \in \{1, \ldots, K-1\}, p, X_{i,j,f,k,p} = 1 \tag{5}$$

$$PS_{i,1} = SC_{f,k,p}, \quad \forall i, f, k \in \{1, \ldots, K-1\}, p, X_{i,j,f,k,p} = 1 \tag{6}$$

$$SS_{f,k,p} = MC_{f,k,p-1}, \quad \forall f, k \in \{1, \ldots, K-1\}, p \in \{2, \ldots, P\} \tag{7}$$

Proof. According to (2) and (3), E_i and T_i are calculated by $PC_{i,6}$ and d_i, where due date d_i is the fixed value provided by the scheduling problem. Furthermore, $PC_{i,6}$ is decided by $(PS_{i,6}-p_{i,6,f,k})$, for the scheduled solution, $PS_{i,6}$ is the only related parameter to E_i and T_i. Eq. (7) means that the machine setup is executed right after the completion of the predecessor operation on the same machine, so setup time is fixed. By Eqs. (5) and (6), $PS_{i,6}$ is fixed once the setup time is confirmed. Therefore, the E_i and T_i cannot be decreased if the scheduling solution is not changed.

Property 2. *If two consecutive operations performed on one machine, $O_{i1,j1}$ and $O_{i2,j2}$, are from one family, the later operation $O_{i2,j2}$ does not consume setup time and electricity.*

Proof. According to the description of the DFJSP-PC in Section 3, the setup stage is considered when two consecutive operations from jobs of different families are performed on one machine. $O_{i1,j1}$ and $O_{i2,j2}$ are from one family, so the setup time and setup electricity is not consumed.

4. Methodology

4.1. Algorithm framework

The framework of the proposed algorithm is illustrated in Fig. 2. Firstly, the population is strategically initialized in Section 4.4, where the encoding and decoding are defined in Sections 4.2 and 4.3, respectively. Then, RL component searches the solutions by selecting appropriate actions under various scheduling circumstances in Section 4.5. Next, the non-dominated sorting and crowding distance sorting are used to retain the superior solutions of the population. Finally, two solution refinement strategies further improve the solutions produced by RL component, which is stated in Section 4.6. In Fig. 2, N denotes the population size, where $N = 100$; N and $2N$ on arrows show the population size after the corresponding steps.

4.2. Encoding

This study utilizes three vectors named family distribution vector (FV), scheduling vector (SV) and machine assignment vector (MV) to describe the solution of the considered DFJSP-PC.

In a solution of the considered DFJSP-PC: (1) the family distribution is expressed by FV, where each element is the distributed factory number of the corresponding family; (2) the operation order in each factory is described by SV, where each element means the job number, and the jth occurrence of the job number is the jth operation of the

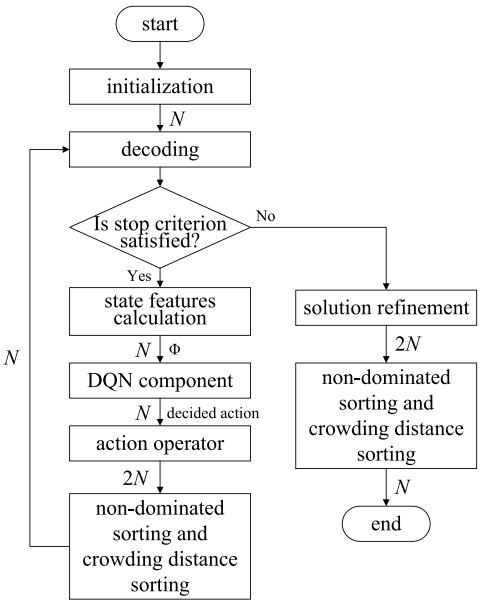


Fig. 2. Algorithm framework.

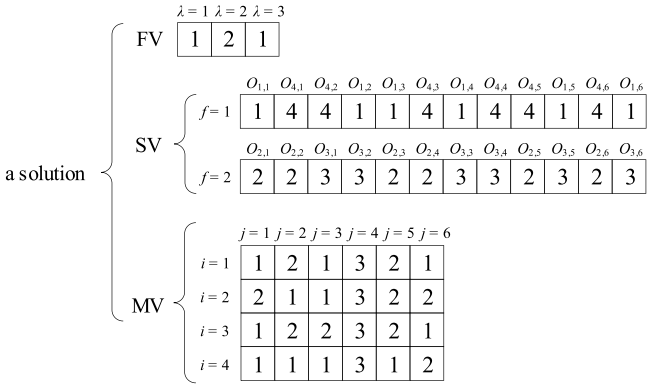


Fig. 3. Solution representation.

job; (3) the machine assignment of all operations is arranged in MV, where the element of operation $O_{i,j}$ is the assigned machine number.

The solution representation of a 2-factory, 3-machine, and 5-job instance (2F3M5J) is shown in Fig. 3. In the instance, three families are included, family 1 contains job 1, family 2 contains jobs 2 and 3, and family 3 contains job 4. According to Fig. 3, jobs of families 1 and 3 are assigned to factory 1, and jobs of family 2 are assigned to factory 2. In MV, all concrete curing operations are assigned the last machine in their factories.

4.3. Decoding

The objectives of a solution can be calculated once the completed solution is provided. The corresponding processing times and setup times of Fig. 3 is listed in Tables 1 and 2, respectively. From Table 1, jobs 2 and 3 are in family 2, and jobs 1 and 4 are in families 1 and 3, respectively; the due date of each job is provided, and the processing times of all operations at all machines in all factories are listed. The "–" indicates that the machine cannot perform the corresponding operation. For example, if operation $O_{1,1}$ is assigned at machine 1 of factory 1, the processing time is 3 units. Table 2 lists all the setup times between the adjacent performed jobs at one machine in all scenarios. For instance, the setup time between the jobs from family 1 and the jobs from family 2 is 0.5 unit. The "0" in Table 2 means no setup time is needed in jobs from one family. The black fonts in Table 1 shows the assigned machines of all operation in Fig. 3. According to operation sequence in SV, all operations in families are performed by their assigned machines in MV of the assigned factories in FV. Setup time is considered when two consecutive operations from different families are performed on one machine. After the processing times of all operations are confirmed, the TWET and TEC can be calculated by (1) and (4), respectively. Fig. 4 is the Gantt chart of the solution in Fig. 3, where the family distribution, operation sequence, and machine assignment are illustrated.

4.4. Initialization

Appropriate initialization strategies can improve the optimization efficiency, specifically in the starting stage of the iterations. In this study, two initialization strategies are designed to improve the performance of the proposed algorithm.

To decrease the setup time and energy consumption, Property 2 and FIFO (first in and first out) dispatching rule (Lin et al., 2019) are used to design the initialization strategy called *LowSetup*. The *LowSetup* strategy is stated in Algorithm 1. *LowSetup* initialization strategy is applied to generate 2/3 of initial solutions. To enhance the diversity of the population, random strategy is applied to generate 1/3 of initial solutions.

Algorithm 1 Initialization strategy: *LowSetup*

Input: problem data
Output: initial solution
Randomly distribute families into different factories ▷ family distribution on FV
for each factory **do** ▷ operation scheduling on SV
  Randomly select a job from assigned families in this factory
  if the job is unfinished **then**
    Schedule the job until the last operation of the job is assigned
  else
    Randomly select another unfinished job
  end if
end for
for each job i **do** ▷ machine assignment on MV
  for each operation $O_{i,j}$ **do**
    if $j == 1$ **then**
      Randomly assigned $O_{i,1}$ to an available machine
    else if $j == 4$ **then**
      Assign $O_{i,4}$ to concrete curing machine
    else
      Assign $O_{i,j}$ to the same machine as $O_{i,1}$
    end if
  end for
end for

4.5. RL component

The overall RL-based optimization process in this study is organized as follows. At the beginning, the proposed algorithm calculates (or observes) the initial state of the scheduling environment; then, according to the initial state, the algorithm need decide which action (or optimization strategy) should be selected, which is called decision point; next, the selected action is executed to optimize the scheduling solutions; at last, the reward of the selected action is evaluated and the state of the changed scheduling environment should be updated for next decision point.

The details of all parts, i.e., states, actions, reward, and policy method, in RL component are expressed as follows.

Table 1
Processing times of decoding example.

Families	Jobs	Due dates	Operations	Processing times					
				Factory 1			Factory 2		
				Machine 1	Machine 2	Machine 3	Machine 1	Machine 2	Machine 3
Family 1	Job 1	35	$O_{1,1}$	**3**	4	–	4	3	–
			$O_{1,2}$	3	**2**	–	3	3	–
			$O_{1,3}$	**2**	3	–	2	4	–
			$O_{1,4}$	–	–	**12**	–	–	13
			$O_{1,5}$	2	**2**	–	2	3	–
			$O_{1,6}$	**2**	3	–	3	3	–
Family 2	Job 2	40	$O_{2,1}$	5	4	–	5	**5**	–
			$O_{2,2}$	2	2	–	**2**	3	–
			$O_{2,3}$	6	5	–	**5**	6	–
			$O_{2,4}$	–	–	15	–	–	**14**
			$O_{2,5}$	3	4	–	3	**4**	–
			$O_{2,6}$	4	4	–	4	**3**	–
	Job 3	40	$O_{3,1}$	5	4	–	**4**	4	–
			$O_{3,2}$	3	2	–	2	**3**	–
			$O_{3,3}$	6	5	–	5	**5**	–
			$O_{3,4}$	–	–	12	–	–	**12**
			$O_{3,5}$	3	4	–	4	**3**	–
			$O_{3,6}$	3	4	–	**3**	3	–
Family 3	Job 4	45	$O_{4,1}$	**3**	4	–	3	4	–
			$O_{4,2}$	**4**	3	–	2	3	–
			$O_{4,3}$	**3**	3	–	2	3	–
			$O_{4,4}$	–	–	**13**	–	–	12
			$O_{4,5}$	**5**	4	–	4	5	–
			$O_{4,6}$	3	**4**	–	3	3	–

Table 2
Setup times between families of decoding example.

Families in corresponding factory	Setup times					
	Factory 1			Factory 2		
	Family 1	Family 2	Family 3	Family 1	Family 2	Family 3
Family 1	0	0.5	1	0	1	0.5
Family 2	0.5	0	0.5	1	0	1
Family 3	1	0.5	0	0.5	1	0

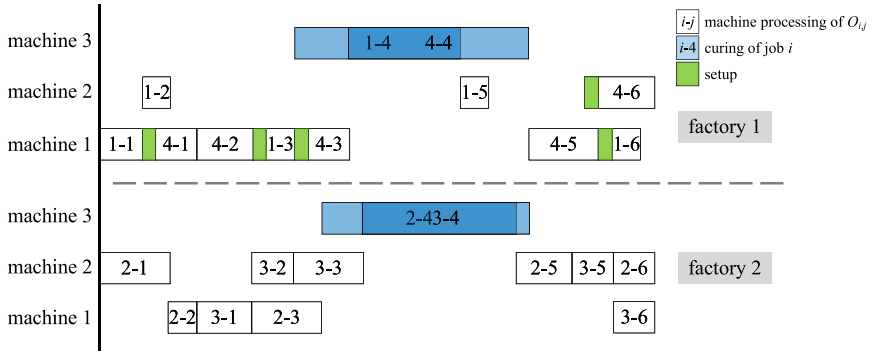


Fig. 4. Gantt chart of solution decoding.

4.5.1. States

In this study, 46 state features are considered to better describe the optimization situation of each solution. The state features can be divided into four types.

The first type of state features are with instance information, which are the numbers of jobs (s_1), total operations (s_2), factories (s_3), and families (s_4).

The second type of state features are with objectives, which include current TWET (s_5), current TEC (s_6), maximum TWET of all factories (s_7), maximum TEC of all factories (s_8), standard deviation TWET of all factories (s_9), standard deviation TEC of all factories (s_{10}), critical TWET proportion (the rate of maximum TWET to the sum TWET of all factories, s_{11}), and critical TEC proportion (the proportion of maximum TEC to the sum TEC of all factories, s_{12}).

The third type of state features are with constraints, which contain total setup time (s_{13}), maximum setup time of all factories (s_{14}), standard deviation setup time of all factories (s_{15}), critical setup time proportion (the rate of maximum setup time to the sum setup times of all factories, s_{16}).

The fourth type of state features are with scheduling process, which include current iteration number (s_{17}), local DQN used proportion (s_{18}), and global DQN used proportion (s_{19}), current total processing time

Table 3
Calculation and initial value definition of state features s_1–s_{23}.

State features	Calculation at decision point t	Initial values $s_{t=0}$
Number of jobs s_1	I	I
Number of total operations s_2	$\sum_{i=1}^{I} OP_i$	$\sum_{i=1}^{I} OP_i$
Number of factories s_3	F	F
Number of families s_4	Λ	Λ
Current TWET s_5	See (1)	From initial solution
Current TEC s_6	See (4)	From initial solution
Maximum TWET of all factories s_7	$\max\{TWET_f\}, \forall f$	From initial solution
Maximum TEC of all factories s_8	$\max\{TEC_f\}, \forall f$	From initial solution
Standard deviation TWET of all factories s_9	$\text{std}\{TWET_f\}, \forall f$	From initial solution
Standard deviation TEC of all factories s_{10}	$\text{std}\{TEC_f\}, \forall f$	From initial solution
Critical TWET proportion s_{11}	$\max\{TWET_f\}/TWET, \forall f$	From initial solution
Critical TEC proportion s_{12}	$\max\{TEC_f\}/TEC, \forall f$	From initial solution
Total setup time s_{13}	$\sum_{f=1}^{F}\sum_{k=1}^{K}\sum_{p=1}^{P}\sum_{m=1}^{M} sp_{f,k,p,m}$	From initial solution
Maximum setup time of all factories s_{14}	$\max\{\sum_{k=1}^{K}\sum_{p=1}^{P}\sum_{m=1}^{M} sp_{f,k,p,m}\}, \forall f$	From initial solution
Standard deviation setup time of all factories s_{15}	$\text{std}\{\sum_{k=1}^{K}\sum_{p=1}^{P}\sum_{m=1}^{M} sp_{f,k,p,m}\}, \forall f$	From initial solution
Critical setup time proportion s_{16}	$\frac{\max_f\{\sum_{k=1}^{K}\sum_{p=1}^{P}\sum_{m=1}^{M} sp_{f,k,p,m}\}}{\sum_{f=1}^{F}\sum_{k=1}^{K}\sum_{p=1}^{P}\sum_{m=1}^{M} sp_{f,k,p,m}}, \forall f$	From initial solution
Current iteration number s_{17}	t	0
Local DQN used proportion s_{18}	t_{local}/t	0
Global DQN used proportion s_{19}	t_{global}/t	0
Current total processing time s_{20}	$\sum_{f=1}^{F}\sum_{k=1}^{K}\sum_{p=1}^{P}\sum_{m=1}^{M} tp_{f,k,p,m}$	From initial solution
Maximum processing time of all factories s_{21}	$\max\{\sum_{k=1}^{K}\sum_{p=1}^{P}\sum_{m=1}^{M} tp_{f,k,p,m}\}, \forall f$	From initial solution
Standard deviation processing time of all factories s_{22}	$\text{std}\{\sum_{k=1}^{K}\sum_{p=1}^{P}\sum_{m=1}^{M} tp_{f,k,p,m}\}, \forall f$	From initial solution
Critical processing time of all factories s_{23}	$\frac{\max_f\{\sum_{k=1}^{K}\sum_{p=1}^{P}\sum_{m=1}^{M} tp_{f,k,p,m}\}}{\sum_{f=1}^{F}\sum_{k=1}^{K}\sum_{p=1}^{P}\sum_{m=1}^{M} tp_{f,k,p,m}}, \forall f$	From initial solution

(s_{20}), maximum processing time of all factories (s_{21}), standard deviation processing time of all factories (s_{22}), critical processing time proportion (the proportion of maximum processing time to the sum processing times of all factories, s_{23}).

For all the above state features s_1–s_{23}, another 23 corresponding state features s_{24}–s_{46} at last decision point are also considered. The calculation and initial value definition of s_1–s_{23} are listed in Table 3, where $\max\{\cdot\}$ and $\text{std}\{\cdot\}$ are the functions to calculate the maximum and standard deviation values, respectively; and t_{local} and t_{global} are the used times of local DQN and global DQN, respectively. At initial state, the current iteration number s_{17} is 0; the local and DQN used proportions s_{18} and s_{19} are ruled as 0. State s_1–s_4 are constant. Other values of states are calculated from the initial solution generated by initialization in Section 4.4.

All state features are managed by a one-dimensional vector $\phi = \{s_1, s_2, \ldots, s_{46}\}$. ϕ_t means all the state features value at decision point t. When $t = 0$, both s_N and s_{N+23} ($N \in \{1, 2, \ldots, 23\}$) are equal to the initial state value.

4.5.2. Actions

17 operators as actions (a_1–a_{17}) are designed to optimize the scheduling solution in RL-based iterations. Actions can be divided into two types as global and local. Local actions (a_1–a_{11}) only adjust the operations in one factory, and the global actions (a_{12}–a_{17}) change the operations in multiple factories. The details of all the actions are as follows.

a_1–a_4 search the solution in a random factory on two positions P_1 and P_2 in SV. For instance, the original SV in the factory is $\{1, 2, 3, 4, 5\}$, P_1 is at "1", and P_2 is at "4". a_1 swaps P_1 and P_2, the swapped SV is $\{4, 2, 3, 1, 5\}$. a_2 reverses all the operations between P_1 and P_2, the reversed SV is $\{4, 3, 2, 1, 5\}$. a_3 moves the element at P_2 to the position in front of P_1, the adjusted SV is $\{4, 1, 2, 3, 5\}$. a_4 moves the element at P_1 to the position after P_2, the adjusted SV is $\{2, 3, 4, 1, 5\}$.

a_5–a_7 find the solution in a random factory by changing MV. a_5 changes the machine assignment of $O_{i,j}$ ($j \neq 6$) to another machine k ($k \neq K$). a_6 exchanges the machine assignment of $O_{i',j'}$ and $O_{i'',j''}$ ($j', j'' \neq 6$). a_7 moves $O_{i,j}$ from the machine with most assigned operations to the least one.

a_8–a_{11} search the solution in a random factory on four positions P_1, P_2, P_3 and P_4 in SV. For example, the original SV in the factory is $\{1, 2, 3, 4, 5, 6, 7, 8\}$, P_1 is at "1", P_2 is at "3", P_3 is at "5", and P_4 is at "8". a_8 swaps P_1 and P_2, P_3 and P_4, the swapped SV is $\{3, 2, 1, 4, 8, 6, 7, 5\}$. a_9 reverses all operations between P_1 and P_2, P_3 and P_4, the reversed SV is $\{3, 2, 1, 4, 8, 7, 6, 5\}$. a_{10} means execute a_3 two times at the four positions, the adjusted SV is $\{3, 1, 2, 4, 8, 5, 6, 7\}$. a_{11} means execute a_4 two times at the four positions, the adjusted SV is $\{2, 3, 1, 4, 6, 7, 8, 5\}$.

a_{12}–a_{17} change the family distribution of the solution. a_{12} and a_{13} moves a family of jobs to another factory. a_{14} and a_{15} exchange two families to different factories. a_{16} and a_{17} move a family of jobs from the factories with most distributed operations to the least one. The a_{12}, a_{14}, and a_{16} put all the moved operations to the end of the SV, while the a_{13}, a_{15}, and a_{17} move the operations to the start of the SV.

The numerous random actions provide flexibility in iteration for larger exploration space, preventing the search being trapped into local optima. The knowledge-based actions can guide the model find better solution with the assistance of problem properties. If the model depends much on knowledge-based actions, most solutions in population will tend to be identical, leading poor optimization performance; therefore, in this study, actions contain random strategies and knowledge-based strategies.

4.5.3. Reward

To measure the reward of the considered Pareto-based multi-objective problems, this study design the reward as follows.

TWET_t and TEC_t denote the TWET and TEC objective values at optimization iteration point t, after executing action, the objective values become to TWET_{t+1} and TEC_{t+1}. Fig. 5 depicts the reward calculation process. In Fig. 5, if the action improves the solution (with state features ϕ_{t+1}) in both TWET and TEC, the reward r_t is the positive value of the green area, where $r_t = (\text{TWET}_{ref} - \text{TWET}_{t+1})(\text{TEC}_{ref} - \text{TEC}_{t+1}) - (\text{TWET}_{ref} - \text{TWET}_t)(\text{TEC}_{ref} - \text{TEC}_t)$; if the action deteriorates the solution (with state features ϕ'_{t+1}) in both TWET and TEC, r_t is the negative

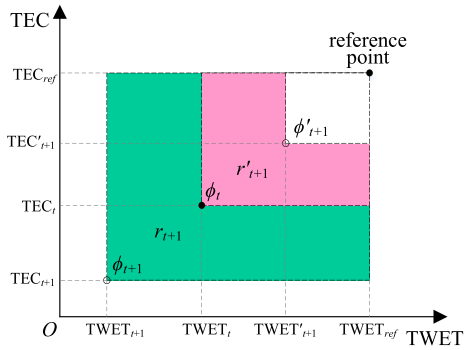


Fig. 5. Reward calculation. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

value of the red area, where $r_t = (\mathrm{TWET}_{ref} - \mathrm{TWET}'_{t+1})(\mathrm{TEC}_{ref} - \mathrm{TEC}'_{t+1}) - (\mathrm{TWET}_{ref} - \mathrm{TWET}_t)(\mathrm{TEC}_{ref} - \mathrm{TEC}_t)$; otherwise, r_t is 0.

In calculating the reward, all objective values should be normalized, in which $\mathrm{TWET}_t = \mathrm{TEC}_t = 1$, and $\mathrm{TWET}_{ref} = \mathrm{TEC}_{ref} = 3$.

4.5.4. Policy network and training method

The design of the proposed DQN model is based on the single DQN model for scheduling problem. To better solve the considered DFJSP-PC, we designed three DQNs to give different types of actions in various situations. For distributed scheduling problems, operators and strategies can be roughly divided into local actions and global actions. The local actions only search the solution by changing the scheduling in only one factory, and the global actions adjust the distribution of different jobs in all factories. The local actions and global actions contribute differently in different optimization stages. To better optimization, we set two different DQNs, i.e., local DQN and global DQN, as policies to give guidance to different types of actions. The selection of the two DQNs is executed by another DQN called selection DQN.

The policy network architecture in the RL component is illustrated in Fig. 6. In Fig. 6, three DQNs, i.e., selection DQN, local DQN, and global DQN, are included, and all the DQNs are fully-connected networks.

Selection DQN has 46 input nodes, 2 output nodes, and three 100-node hidden layers. Local DQN and global DQN have the same 48 input nodes and two 100-node hidden layers. Local DQN has 11 output nodes which represent the 11 local actions (a_1–a_{11}), while global DQN has 6 output nodes that correspond to the 6 global actions (a_{12}–a_{17}). The activation functions of the input, hidden, and output layers are "ReLu", "ReLu", and "purelin", respectively.

All three DQNs include their own online networks (Q_s, Q_l, and Q_g, where s, l, and g denotes selection, local and global DQNs, respectively) and target networks ($\hat{Q}_s$, $\hat{Q}_l$, and $\hat{Q}_g$) for better performance, each pair of networks have the same architectures. The update and training method of the DQNs are expressed in Algorithm 2, where ϵ-greedy policy is utilized. An episode is the whole iteration process that the population is evolved by RL component. In DQN training, each combination of states and actions are sampled as a mini-batch from buffer D. The buffer D contains the combinations $\{\phi_t, a_t, r_t, \phi_{t+1}\}$ at all decision points in training process, where the combinations from the beginning to ending in optimization process can be sampled in network training. In each decision point, mini-batch size combinations of state, actions, and rewards are randomly sampled from buffer D to train the DQN. In DQN training and optimization, according to the scheduling solution, state values can have any possible solutions. The large solution space means large states ranges. In any one state, all actions can be selected to execute as optimization strategy in each iteration. With the ϵ-greedy policy, wider solution space can be searched, providing states and actions pairs in training data. The loss functions of the Q_s, Q_l, and Q_g in DQN training are $\sum_a(y_{j,s} - Q_s(\phi_{j,s}, a_{j,s}; \theta_s))^2$, $\sum_a(y_{j,l} - Q_l(\phi_{j,l}, a_{j,l}; \theta_l))^2$, and $\sum_a(y_{j,g} - Q_g(\phi_{j,g}, a_{j,g}; \theta_g))^2$, respectively, where the meaning of $y_{j,s}$, $y_{j,l}$, and $y_{j,g}$ are defined in the 25th, 27th, and 29th lines of Algorithm 2.

The parameters setting in training process are listed in Table 4, where the size of population, the parameters of selection, local, and global DQNs are provided. The population size is determined by the experience of algorithm design, which is similar to researches (Du et al., 2021, 2022a,b; Li et al., 2022b,a; Yang et al., 2016; Wang and Wang, 2021; Zhao et al., 2020; Jiang and Wu, 2021; Anvari et al., 2016; De Giovanni and Pezzella, 2010; Wang et al., 2021; Chang and Liu, 2017; Wu et al., 2019); the size of mini-batch is similar as Mnih et al. (2015) and Luo (2020); the discount factors are referred from Luo (2020); the learning rate of all DQNs, τ_s, τ_g, and τ_l in soft target update

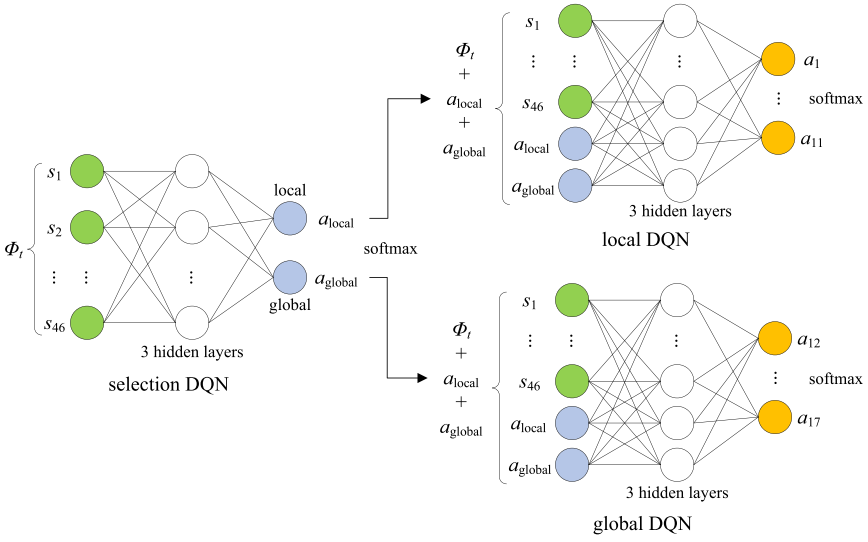


Fig. 6. Network topology of RL component.

Algorithm 2 DQNs training

Input: training parameters
Output: trained Q_s, Q_l, Q_g, $\hat{Q}_s$, $\hat{Q}_l$, and $\hat{Q}_g$

Initialize a population with L solutions (c.f. Section 4.4), each solution experience a whole episode of optimization
Initialize random weights of networks Q_s, Q_l, and Q_g with corresponding weights θ_s, θ_l, and θ_g
Give the weights of networks $\hat{Q}_s$, $\hat{Q}_l$, and $\hat{Q}_g$, where $\theta_s = \hat{\theta}_s$, $\theta_l = \hat{\theta}_l$, and $\theta_g = \hat{\theta}_g$ are satisfied
for each episode **do**
  for each optimization iteration point $t = 0, ..., T$ **do**
    Observe current state features ϕ_t of the scheduling environment
    Select an action $a_{t,s} \in \{a_l, a_g\}$ with ϵ-greedy policy by Q_s according to ϕ_t
    Observe state features $\phi'_t = \{\phi_t, a_l, a_g\}$
    if $a_{t,s} == a_l$ **then** ▷ this iteration selects local strategies
      Execute an action $a_{t,l} \in \{a_1, ..., a_{11}\}$ with ϵ-greedy policy by Q_l according to ϕ'_t
      Calculate the corresponding reward $r_{t,l}$ and observe new state features ϕ_{t+1} and ϕ'_{t+1}
      Store the combination $\{\phi'_t, a_{t,l}, r_{t,l}, \phi'_{t+1}\}$ in D_l
      Store the combination $\{\phi_t, a_{t,s}, r_{t,l}, \phi_{t+1}\}$ in D_s
    else ▷ this iteration selects global strategies
      Execute an action $a_{t,g} \in \{a_{12}, ..., a_{17}\}$ with ϵ-greedy policy by Q_g according to ϕ'_t
      Calculate the corresponding reward $r_{t,g}$ and observe new state features ϕ_{t+1} and ϕ'_{t+1}
      Store the combination $\{\phi'_t, a_{t,g}, r_{t,g}, \phi'_{t+1}\}$ in D_g
      Store the combination $\{\phi_t, a_{t,s}, r_{t,g}, \phi_{t+1}\}$ in D_s
    end if
    if the capacities of D_s, D_l, and D_g all larger than the minibatch size **then**
      Sample random minibatch of combinations $\{\phi_{j,s}, a_{j,s}, r_{j,s}, \phi_{j+1,s}\}$ from D_s
      Sample random minibatch of combinations $\{\phi_{j,l}, a_{j,l}, r_{j,l}, \phi_{j+1,l}\}$ from D_l
      Sample random minibatch of combinations $\{\phi_{j,g}, a_{j,g}, r_{j,g}, \phi_{j+1,g}\}$ from D_g
      for each group of combinations of the minibatches **do**
        Set $y_{j,s} = \begin{cases} r_{j,s}, \text{if episode terminates at step } j+1 \\ r_{j,s} + \gamma_s \hat{Q}_s(\phi_{j+1,s}, \arg\max_{a'} Q_s; \theta'_s), \text{otherwise} \end{cases}$
        Perform a gradient descent on loss $\sum_a (y_{j,s} - Q_s(\phi_{j,s}, a_{j,s}; \theta_s))^2$ with respect to the θ_s of Q_s
        Set $y_{j,l} = \begin{cases} r_{j,l}, \text{if episode terminates at step } j+1 \\ r_{j,l} + \gamma_l \hat{Q}_l(\phi_{j+1,l}, \arg\max_{a'} Q_l; \theta'_l), \text{otherwise} \end{cases}$
        Perform a gradient descent on loss $\sum_a (y_{j,l} - Q_l(\phi_{j,l}, a_{j,l}; \theta_l))^2$ with respect to the θ_l of Q_l
        Set $y_{j,g} = \begin{cases} r_{j,g}, \text{if episode terminates at step } j+1 \\ r_{j,g} + \gamma_g \hat{Q}_g(\phi_{j+1,g}, \arg\max_{a'} Q_g; \theta'_g), \text{otherwise} \end{cases}$
        Perform a gradient descent on loss $\sum_a (y_{j,g} - Q_g(\phi_{j,g}, a_{j,g}; \theta_g))^2$ with respect to the θ_g of Q_g
        Every C steps update target network by $\hat{Q}_s = \tau_s Q_s + (1 - \tau_s)\hat{Q}_s$ ▷ soft target weight update
        Every C steps update target network by $\hat{Q}_l = \tau_l Q_l + (1 - \tau_l)\hat{Q}_l$
        Every C steps update target network by $\hat{Q}_g = \tau_g Q_g + (1 - \tau_g)\hat{Q}_g$
      end for
    end if
  end for
end for

Table 4
Parameter setting in DQNs training.

Parameters	Values
Population size	100
Learning rate of all DQNs	0.005
τ_s, τ_g, and τ_l in soft target update strategy	0.05
C in soft target update strategy	10
Size of mini-batch	32
Discount factors γ_s, γ_g, and γ_l	0.9

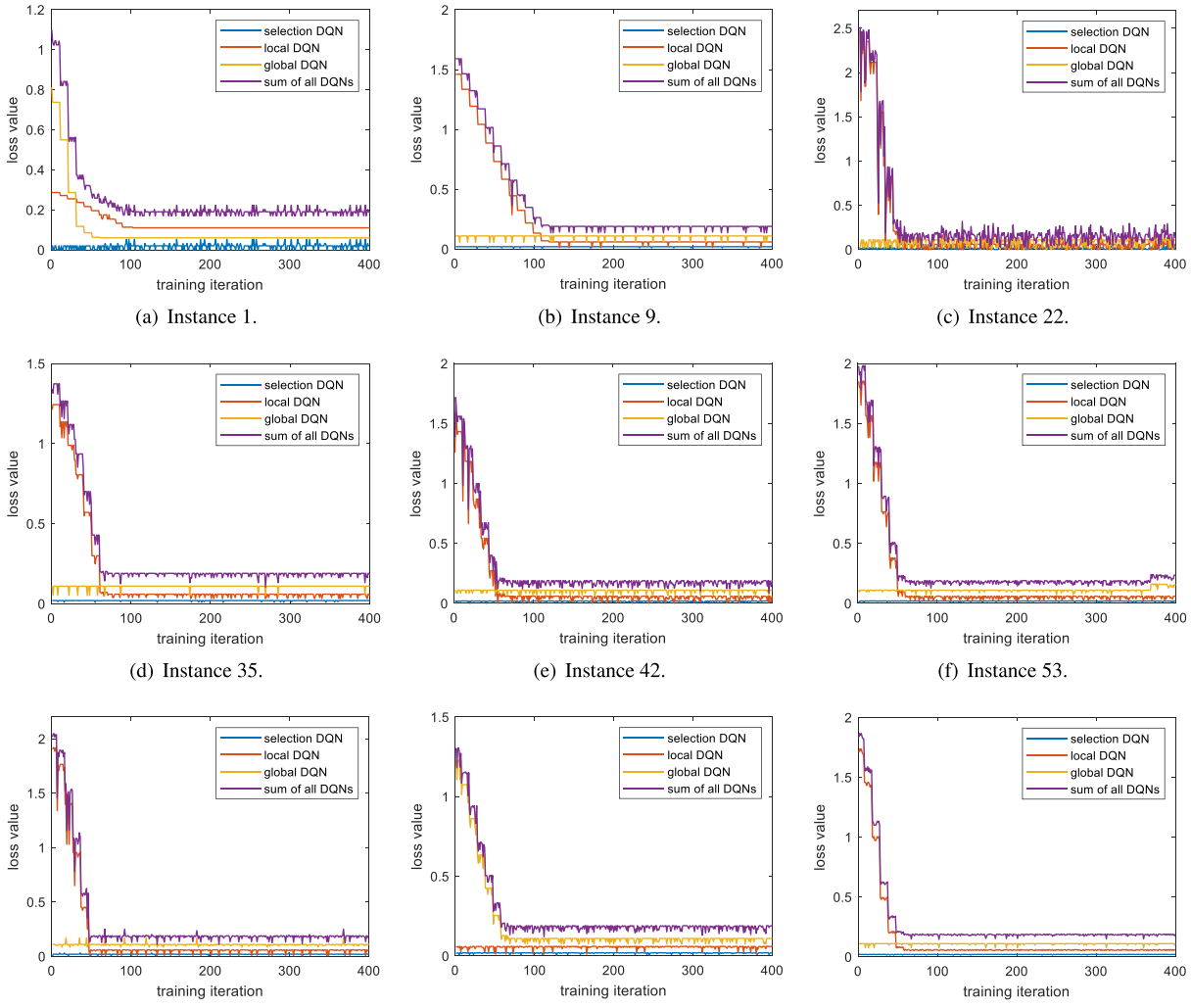
strategy, and C in soft target update strategy are decided by experience. Other key parameters in DQN-EA-SR are calibrated in Section 5.3.

The loss values of the selection DQN, local DQN, and global DQN in network training of nine representative instances are drawn in Figs. 7(a)–7(i). From Fig. 7, the losses of local DQN and global DQN converged in starting stages of training, and the loss of selection DQN fluctuates within low level, which can be accepted. The training time is determined by epoch, episode, and problem scale. The epoch and episode are related to the numbers of iterations, and the problem scale is to decoding time. Specifically, for small-scaled DFJSP-PC with 10 jobs, the training time is less than 40 s in C++ environment; for large-scaled problem with 300 jobs, the training is more than 1 h. Once the model is trained, it can be directly used in scheduling. In the training of the proposed reinforcement leaning model, the parameters of the DQNs are updated at every epochs of every episodes. In this study, the epoch and episode numbers are 1000 and 30, respectively, which means the proposed DQNs model is updated 30 000 times in an instance.

4.6. Solution refinement

After being optimized by the *sequence-to-sequence* RL component, the scheduling solutions have high quality. To further improve the solution quality and population diversity, two refinement strategies are proposed as follows.

In the decoding, all operations are performed as early as possible; however, the starting time of the processing time can be adjusted for better decreasing the objectives, i.e., TWET and TEC. We propose two



(a) Instance 1. (b) Instance 9. (c) Instance 22.

(d) Instance 35. (e) Instance 42. (f) Instance 53.

(g) Instance 66. (h) Instance 75. (i) Instance 89.

Fig. 7. Loss values in DQNs training.

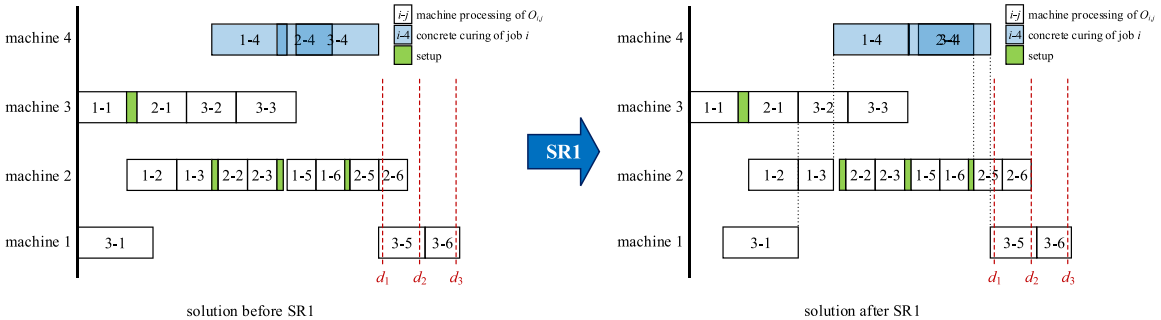


Fig. 8. Solution refinement strategy SR1.

different refinement strategies, i.e., SR1 and SR2, to improve the solutions. SR1 is to decrease the TWET by changing the starting processing time of all operations to cut down on the earliness and tardiness as much as possible. SR2 is to decrease the TEC by moving the operation into low electricity price intervals as much as possible. The details of SR1 and SR2 are expressed in Algorithms 3 and 4, respectively. SR1 and SR2 are designed based on Property 1.

Fig. 8 illustrates the SR1 in a single factory including 4 machines and 3 jobs, in which job 1 is from a family, and jobs 2 and 3 are from another family. In Fig. 8, according to Property 1, the tardiness penalties of jobs 1 and 3 cannot be decreased, only the earliness penalty of job 2 can be decreased by moving the operation $O_{2,6}$ backward; then, according to Algorithm 3, all the processing times of other related operations are changed. From Fig. 8, SR1 can further decrease the TWET based on the solutions optimized by the RL component.

Fig. 9 shows the SR2 in a single factory, whose information is the same as that in Fig. 8. With SR2, the $O_{2,5}$ in Fig. 9 moves into low electricity areas to decrease the TEC; then, $O_{1,6}$ follows $O_{2,5}$, which

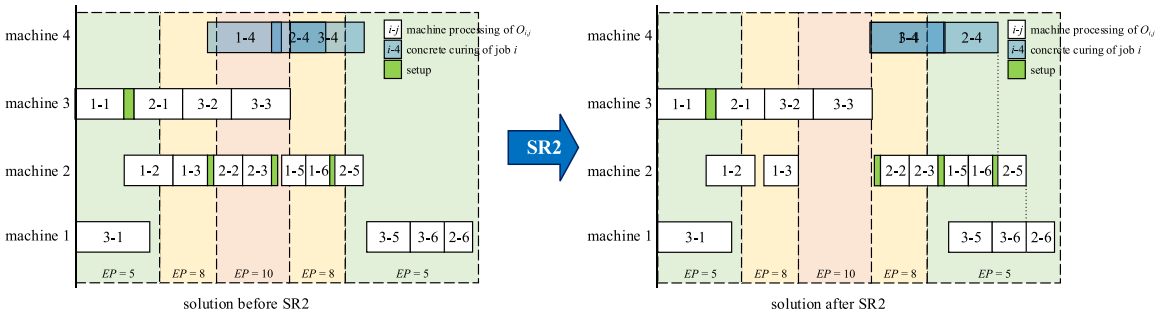


Fig. 9. Solution refinement strategy SR2.

Algorithm 3 SR1: decrease TWET

Input: solution x, problem data
Output: solution x' refined by SR1

for each factory f **do**
  Sort all the operations assigned in factory f according to their completion times $MC_{i,j}$
  All the sorted operations are managed in set X by descending $MC_{i,j}$ order
  Initialize maximum completion time of all machines $maxCT_{f,k} = +\infty, \forall f, k$
  for each operation $O_{i,j}$ in X (from maximum to minimum $MC_{i,j}$) **do**
    if $j == 6$ **then** ▷ the last operation of the job
      if Oi, j is completed before its due date d_i **then**
        Move the processing times by $MC_{i,j} = \min\{d_i, maxCT_{f,k}\}$ and $MS_{i,j} = MC_{i,j} - p_{i,j,f,k}$
        Move the setup times by $SC_{i,j} = MS_{i,j}$ and $SS_{i,j} = SC_{i,j} - s_{f,k,\lambda_1,\lambda_2}$
      end if
      Update $maxCT_{f,k} = SS_{i,j}$
    else if $j == 4$ **then** ▷ the curing operation
      Move the processing times by $MC_{i,j} = MS_{i,j+1}$ and $MS_{i,j} = MC_{i,j} - p_{i,j,f,k}$
      Move the setup times by $SC_{i,j} = MS_{i,j}$ and $SS_{i,j} = SC_{i,j} - s_{f,k,\lambda_1,\lambda_2}$
    else ▷ other operations
      Move the processing times by $MC_{i,j} = \min\{MS_{i,j+1}, maxCT_{f,k}\}$ and $MS_{i,j} = MC_{i,j} - p_{i,j,f,k}$
      Move the setup times by $SC_{i,j} = MS_{i,j}$ and $SS_{i,j} = SC_{i,j} - s_{f,k,\lambda_1,\lambda_2}$
      Update $maxCT_{f,k} = SS_{i,j}$
    end if
  end for
end for
Calculate the objectives, i.e., TWET and TEC, of the refined solution x'

Algorithm 4 SR2: decrease TEC

Input: solution x
Output: solution x'' refined by SR2

for each factory f **do**
  Sort all the operations assigned in factory f according to their completion times $MC_{i,j}$
  All the sorted operations are managed in set X by descending $MC_{i,j}$ order
  Initialize maximum completion time of all machines $maxCT_{f,k} = +\infty, \forall f, k$
  for each operation $O_{i,j}$ in X (from maximum to minimum $MC_{i,j}$) **do**
    Move the processing time of $O_{i,j}$ to the least TEC value, when $MS_{i,j} = minMS$
    if $j == 6$ **then** ▷ the last operation of the job
      $MC_{i,j} = MS_{i,j} + p_{i,j,f,k}$
    else ▷ other operations
      $MC_{i,j} = \min\{MS_{i,j} + p_{i,j,f,k}, MS_{i,j+1}\}$
    end if
    Move the setup times by $SC_{i,j} = MS_{i,j}$ and $SS_{i,j} = SC_{i,j} - s_{f,k,\lambda_1,\lambda_2}$
    if $j \neq 4$ **then**
      Update $maxCT_{f,k} = SS_{i,j}$
    end if
  end for
end for
Calculate the objectives, i.e., TWET and TEC, of the refined solution x''

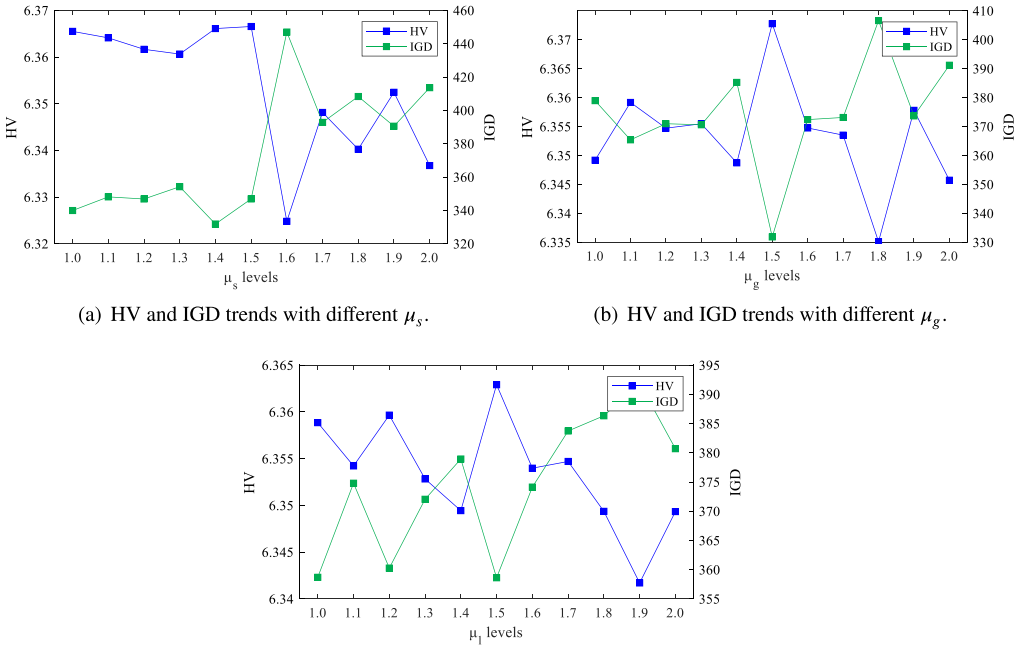
gives more space for $O_{2,5}$ to decrease TEC. From Fig. 9, SR2 can further optimize the TEC after RL component.

5. Experimental analysis

This section gives a series of numerical test of the proposed algorithm with other compared algorithms and strategies. All the compared algorithms and strategies were implemented in C++ on an Intel Core i7 3.4-GHz PC with 16 GB of memory. The CPU running time of each compared algorithm is $(0.6 * OP_i)$ s.

5.1. Training and testing instances

In this study, three groups of instances are employed for experimental analysis. The first group is utilized to validate the proposed mixed integer linear programming (MILP) model, including 24 instances, where the numbers of job are $\{5, 8, 10, 12, 15, 20\}$, the numbers of machines in each factory are $\{3, 5\}$, and the numbers of factory are $\{2, 3\}$. The second group has 90 instances, which are for the testing in performance comparison, where the job numbers are $\{10, 20, 30, 50, 80, 100, 150, 200, 250, 300\}$, the machine numbers in each factory are $\{5, 8, 10\}$, and the factory numbers are $\{2, 3, 5\}$. The third group of instances are designed for network training, where 90 training instances are contained. The scale of each training instance is the combination of the numbers of jobs, machines, and factories. The numbers of job are $\{10, 20, 30, 50, 80, 100, 150, 200, 250, 300\}$; the machine numbers in each factory are $\{5, 8, 10\}$; and the factory numbers are $\{2, 3, 5\}$. The third group of instances have the same job, machine, and factory scales as the instances in the second group, but the processing times and setup times of all operations are different from those in



(a) HV and IGD trends with different μ_s.

(b) HV and IGD trends with different μ_g.

(c) HV and IGD trends with different μ_l.

Fig. 10. Parameters validation. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

the second group. The name of each instance contains the numbers of factories and jobs, for example, "2F3M10J" means the instance includes 2 factories, 3 machines in each factory, and 10 jobs.

In general, the per time unit of tardiness penalty is greater than that of earliness penalty (Ko and Wang, 2010; Yang et al., 2016; Ma et al., 2018); therefore, for all groups of instances in this study, the per time unit of tardiness penalty is 10, while the per time unit of earliness penalty is 2, which is similar to Ma et al. (2018) and Yang et al. (2016). The time of concrete curing is 12, and the time of other processing stages are set in interval [1, 5]. The setup time between different jobs is between 0.5 to 1.0 except concrete curing stage. The due date of each job is generated from a uniform distribution on the interval of $[50 * (i/5 + 1), 50 * (i/5 + 2)]$, where i is job number. It should be noted that although the scales of testing and training instances are the same, the processing times and setup times are generated at random in the corresponding ranges, which means the processing and setup times of the same scale of testing and training instances are different.

5.2. Evaluation criteria

In this study, hypervolume (HV) and inverted generational distance (IGD) are selected to evaluate the performance of all compared algorithms. The HV and IGD can be calculated by Eqs. (8) and (9), respectively.

$$HV = \lambda(\cup_{i=1}^{|NS|} v_i) \tag{8}$$

$$IGD = \frac{1}{|NS|} \sum_{j \in PF^*} \left(\min_{i \in NS} |j - i|\right) \tag{9}$$

where NS denotes the non-dominated set, PF^* means the true Pareto front, v_i is the HV made by solution i and reference point $(3, 3)$, and $\lambda(\bullet)$ is Lebesgue measure.

In this study, two objectives, TWET and TEC should be minimized. If the TWET and TEC of a solution are small, v_i is large, and the HV is large. Furthermore, the larger HV value means that the corresponding non-dominated set has more high-quality solutions. IGD is the sum distance between the solutions of non-dominated set and PF^*. If most solutions in non-dominated set are close in a small area, IGD is large, meaning the diversity of the set is not satisfying. On the contrary, smaller IGD indicates that the solution set is more similar to the PF^*, meaning a better result. In conclusion, larger HV or smaller IGD means better performance.

HV and IGD are commonly used as performance measures in multi-objective optimization. HV can signify all the objectives of all solutions in a non-dominated set, and IGD can reflect the diversity of the non-dominated set. Therefore, HV and IGD are selected as evaluation criteria in this study.

The comparison results of HV and IGD values are evaluated by relative percentage index (RPI), which can be calculated as follow:

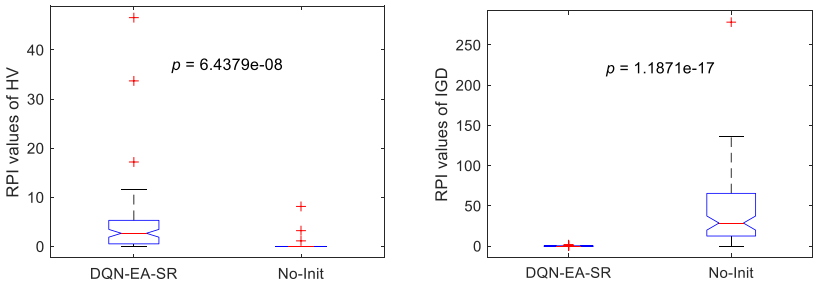
$$RPI = \frac{|f_c - f_b|}{f_b} \times 100, \tag{10}$$

where f_c denotes the HV or IGD value of the current compared algorithm, and f_b is the minimum HV or IGD value in all compared algorithms. According to the definitions of HV and IGD, larger RPI value of HV or smaller that of IGD indicates better performance.

5.3. Parameters calibration

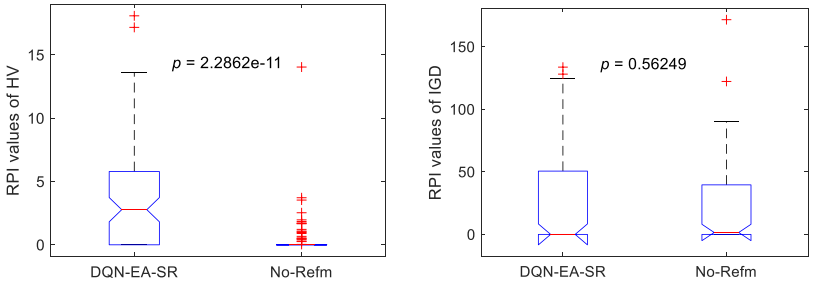
The μ in softmax output selection is essential for the optimization performance, so μ_s, μ_g, and μ_l in selection, global, and local DQNs are calibrated in this study. In this study, all the softmax parameters were calibrated in the same set {1.0, 1.1, 1.2, 1.3, 1.4, 1.5, 1.6, 1.7, 1.8, 1.9, 2.0}, where all combinations of the three parameters were tested in instance 2F5M10J, and the HV and IGD results were collected.

Figs. 10(a)–10(c) illustrate the average HV and IGD trends with different μ_s, μ_g, and μ_l, respectively. In each figure, the HV and IGD values at different μ levels are depicted in blue and green square points, respectively. The appropriate μ should has large HV and small IGD values. According to the figures, when $\mu_s = 1.4$, $\mu_g = 1.5$, and $\mu_l = 1.5$, both HV and IGD can obtain satisfying outputs. The following experiment analyses are based on this parameter setting.



(a) HV results of initialization comparison. (b) IGD results of initialization comparison.

Fig. 11. Comparison results of initialization strategy.



(a) HV results of solution refinement strategies comparison. (b) IGD results of solution refinement strategies comparison.

Fig. 12. Comparison results of solution refinement strategies.

5.4. Comparison with initialization strategy

This section compares the DQN-EA-SR with No-Init to prove the effectiveness of initialization strategies, where No-Init denotes the DQN-EA-SR with random initialization strategy, other parts of the No-Init are the same as the DQN-EA-SR.

In this study, all comparison effectiveness were evaluated by Analysis of Variance (ANOVA). Figs. 11(a) and 11(b) show the RPI values of HV and IGD, respectively. From Fig. 11, the DQN-EA-SR has better HV and IGD results compared with the No-Init. The p-values of all the ANOVA results were smaller than 0.01, indicating the effectiveness of the proposed initialization strategies is significant.

5.5. Comparison with solution refinement strategies

This section gives the comparison with the proposed algorithm without refinement strategies (No-Refm) to quantify the effectiveness of the solution refinement strategies.

The ANOVA results are illustrated in Figs. 12(a) and 12(b), where the p-values show that the difference between the proposed algorithm with No-Refm is significant in HV but not in IGD result. From the comparison, it can be found that the refinement strategies had little impact on IGD; however, the refinement strategies can significantly improve the HV performance. The results indicate that the refinement strategies can optimize the TWET and TEC simultaneously, but may not improve the diversity of non-dominated set.

5.6. Comparison with different DQN topologies

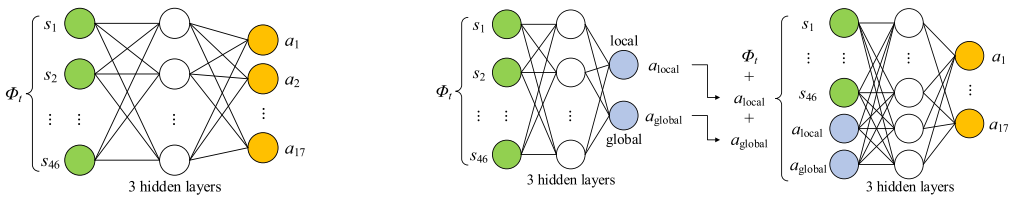
This study proposed a new DQN topology for better performance. To evaluate the priority of this topology, two models, i.e., SingleDQN and DoubleDQN, were considered as comparison. The architectures of the SingleDQN and DoubleDQN are shown in Figs. 13(a) and 13(b), respectively. In Fig. 13(a), the SingleDQN has the same topology of the selection DQN in the proposed algorithm, all state features are as inputs, and all actions are as outputs. The SingleDQN is to test the effectiveness of a_{local} and a_{global}. In Fig. 13(b), the DoubleDQN has a selection DQN and another DQN, 46 state features, a_{local}, and a_{global} are as inputs of the later network, all actions are as outputs. The DoubleDQN is to test the division performance of the local and global actions in DQN-EA-SR.

The ANOVA results of the comparison are shown in Fig. 14. From Fig. 14(a), the HV differences in the three compared DQN structures are not significant. In Fig. 14(b), the IGD results show that the DQN-EA-SR can significantly improve the diversity of the non-dominated solutions compared with other compared DQN topologies. The comparison results indicate that the proposed new DQN topology can partly improve the HV and significantly enhance the population diversity for better optimization.

5.7. Comparison with competitive algorithms

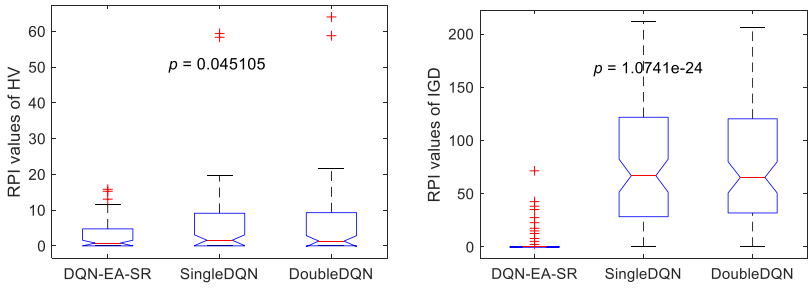
In this section, we selected five algorithms, i.e., EMA proposed by Luo et al. (2020), HMOMA designed by Sang and Tan (2022), SLGA built by Chen et al. (2020), DQN-L constructed by Luo (2020), and HDQN constructed by Li et al. (2022c), to compare the performance in solving the considered problem. In the comparison algorithms, EMA and HMOMA were specifically designed for solving the DFJSP. SLGA was learning-to-improve RL-based algorithm for solving FJSP, where the FJSP and DFJSP are similar except for job distribution constraint. Two end-to-end RL-based algorithms DQN-L and HDQN are for solving constrained FJSP. Because only one solution is generated in end-to-end approach, we collected 100 solutions by each end-to-end approach as a population to calculate HV and IGD values for each instance. The parameter settings of all compared algorithms are listed in Table 5, where optimization objectives, evaluation criteria, and other main information are included.

For the EMA and HMOMA, the researches assumed that the jobs can be transferred between different factories in manufacturing. The SLGA has no other special constraint. To make the comparison algorithms adapt to the FJSP-PC, all compared algorithms assumed that the jobs cannot be transferred in different factories, and all algorithms used the



(a) SingleDQN architecture. (b) DoubleDQN architecture.

Fig. 13. Architectures of comparison DQNs.



(a) HV results of different DQN topologies. (b) IGD results of different DQN topologies.

Fig. 14. Comparison results with different network topologies.

Table 5
Problem information and parameter settings of comparison algorithms.

Algorithms	Objectives	Evaluation criteria	Population size	Strategy probabilities and parameters
EMA (Luo et al., 2020)	Makespan, maximum work-load of factories, and total energy consumption	C-metric, IGD	150	Crossover 0.7, mutation 0.4, local search 0.15
HMOMA (Sang and Tan, 2022)	Makespan, energy consumption, equipment load, delay time, and manufacturing quality	IGD, HV	126	Mating 0.8, mutation 0.1, penalty parameter 5
SLGA (Chen et al., 2020)	Makespan	RPI values of makespan	$5 * K * I$	Learning rate 0.75, discount rate 0.2, initial reward 1, greedy rate 0.85
DQN-L (Luo, 2020)	Total tardiness	Objective value	100 final results as a population	Entropy control parameter 1.6
HDQN (Li et al., 2022c)	Makespan, total energy consumption	RPI value of total weighted objectives	100 final results as a population	Greedy rate 0.6, soft target update rate 0.01

same encoding and decoding components as the proposed DQN-EA-SR. For fairness, the iteration times of all compared algorithms are the same as the DQN-EA-SR. Meanwhile, the PF^* in calculating the IGD objective was obtained by all compared algorithms, guaranteeing the same comparison level in numerical results.

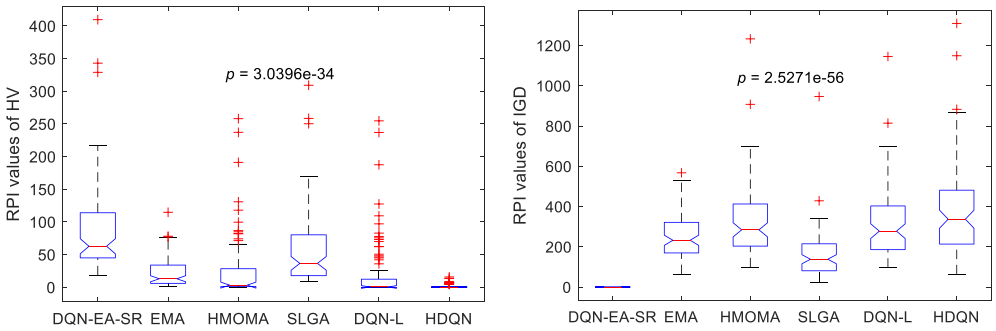
The ANOVA results of the RPI values of HV and IGD are illustrated in Figs. 15(a) and 15(b), respectively. From Figs. 15(a) and 15(b), the proposed DQN-EA-SR outperforms other competitive algorithms both in HV and IGD. The p-values on the RPI values of HV and IGD are both smaller than 0.01, indicating the performance of the proposed DQN-EA-SR is significantly better than other compared algorithms. On average, for HV results, the DQN-EA-SR performs well 27.9%, 32.8%, 16.3%, 34.6% and 41.2% better than EMA, HMOMA, SLGA, DQN-L, and HDQN, respectively; for IGD results, the DQN-EA-SR performs well 232.1%, 308.2%, 204.4%, 311.3%, and 418.7% better than EMA, HMOMA, SLGA, DQN-L, and HDQN, respectively.

5.8. Experiments analysis

This section provides as integrated analysis of all the experiments. The DQN-EA-SR can obtain better solutions of the considered DFJSP, which can be the consequences of the following points. (1) The model can obtain better optimization performance after parameter calibration, where the best parameter sets are obtained in Section 5.3. (2) In initial population design, strategy *LowSetup* that combines problem property with canonical dispatching rule provides a satisfying starting condition in iteration, contributing the optimization improvement in Section 5.4. (3) After iteration of RL component, the refinement strategies SR1 and SR2 are proposed to specifically decrease the TWET and TEC, respectively, which can further improve the HV performance in Section 5.5. (4) The topology of the DQN-EA-SR contains three DQNs, where the global and local actions are separately executed by two different DQNs, and another DQN is designed to control the selection of two different types of actions. From the comparison with other two related network topologies in Section 5.6, the DQN-EA-SR mainly improves the population diversity, which can assist the model find better solution in iteration. And (5) the comparison experiment with other competitive algorithms designed for solving DFJSP and FJSP suggests the satisfying performance of the DQN-EA-SR, see Section 5.7.

6. Conclusion

This study proposes the DQN-EA-SR to solve the DFJSP considering PC manufacturing, where two practical objectives, TWET and total



(a) HV results of comparison algorithms. (b) IGD results of comparison algorithms.

Fig. 15. Comparison results with competitive algorithms.

time-of-use electricity cost are minimized, simultaneously. In DQN-EA-SR, three DQNs are integrated in topology, where the selection DQN decides the type of actions according to state features, then global DQN or local DQN executes the exact action if one of them is selected. The new topology can improve the diversity of population for better iteration. Furthermore, the population initialization strategy based on knowledge and canonical dispatching rule is designed, enabling the RL component has an appropriate starting iteration. After RL component, two solution refinement strategies are designed to further decrease the objectives, enhancing the optimization ability in solving the considered DFJSP. According to the numerical test by a wide range of instances, the proposed DQN-EA-SR can obtain better outputs compared with other competitive algorithms for DFJSP.

In the future work, the research directions will be made on: (1) applying integrated DQN model to other constrained scheduling circumstances; (2) combining complex and realized constraints in the DFJSP-PC; and (3) introducing more realized optimization objectives in constrained scheduling problems.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgment

This work is supported by National Science Foundation of China under Grant 62173216, Yunnan Key Laboratory of Modern Analytical Mathematics and Applications (No. 202302AN360007).

Appendix A. Supplementary data

Supplementary material related to this article can be found online at https://doi.org/10.1016/j.ijpe.2023.109102.

References

Anvari, B., Angeloudis, P., Ochieng, W., 2016. A multi-objective GA-based optimisation for holistic manufacturing, transportation and assembly of precast construction. Autom. Constr. 71, 226–241.

Cao, J., Pan, R., Xia, X., Shao, X., Wang, X., 2021. An efficient scheduling approach for an iron-steel plant equipped with self-generation equipment under time-of-use electricity tariffs. Swarm Evol. Comput. 60, 100764.

Chang, H.-C., Liu, T.-K., 2017. Optimisation of distributed manufacturing flexible job shop scheduling by using hybrid genetic algorithms. J. Intell. Manuf. 28 (8), 1973–1986.

Chen, R., Yang, B., Li, S., Wang, S., 2020. A self-learning genetic algorithm based on reinforcement learning for flexible job-shop scheduling problem. Comput. Ind. Eng. 149, 106778.

Cheng, C.-Y., Pourhejazy, P., Ying, K.-C., Liao, Y.-H., 2021. New benchmark algorithms for no-wait flowshop group scheduling problem with sequence-dependent setup times. Appl. Soft Comput. 111, 107705.

De Giovanni, L., Pezzella, F., 2010. An improved genetic algorithm for the distributed and flexible job-shop scheduling problem. European J. Oper. Res. 200 (2), 395–408.

Du, Y., Li, J.-q., Chen, X.-l., Duan, P.-y., Pan, Q.-k., 2022a. Knowledge-based reinforcement learning and estimation of distribution algorithm for flexible job shop scheduling problem. IEEE Trans. Emerg. Top. Comput. Intell. 1–15. http://dx.doi.org/10.1109/TETCI.2022.3145706.

Du, Y., Li, J., Li, C., Duan, P., 2022b. A reinforcement learning approach for flexible job shop scheduling problem with crane transportation and setup times. IEEE Trans. Neural Netw. Learn. Syst..

Du, Y., Li, J.-q., Luo, C., Meng, L.-l., 2021. A hybrid estimation of distribution algorithm for distributed flexible job shop scheduling with crane transportations. Swarm Evol. Comput. 62, 100861.

Ho, M.H., Hnaien, F., Dugardin, F., 2021. Electricity cost minimisation for optimal makespan solution in flow shop scheduling under time-of-use tariffs. Int. J. Prod. Res. 59 (4), 1041–1067.

Hosseinzadeh, M.R., Heydari, M., Mahdavi Mazdeh, M., 2022. Mathematical modeling and two metaheuristic algorithms for integrated process planning and group scheduling with sequence-dependent setup time. Oper. Res. 1–51.

Jahani, H., Jain, R., Ivanov, D., 2023. Data science and big data analytics: a systematic review of methodologies used in the supply chain and logistics research. Ann. Oper. Res. 1–58.

Jiang, E., Wang, L., 2020. Multi-objective optimization based on decomposition for flexible job shop scheduling under time-of-use electricity prices. Knowl.-Based Syst. 204, 106177.

Jiang, W., Wu, L., 2021. Flow shop optimization of hybrid make-to-order and make-to-stock in precast concrete component production. J. Clean. Prod. 297, 126708.

Kim, T., Kim, Y.-w., Cho, H., 2020. Dynamic production scheduling model under due date uncertainty in precast concrete construction. J. Clean. Prod. 257, 120527.

Kim, T., Kim, Y.-W., Lee, D., Kim, M., 2022. Reinforcement learning approach to scheduling of precast concrete production. J. Clean. Prod. 130419.

Ko, C.-H., Wang, S.-F., 2010. GA-based decision support systems for precast production planning. Autom. Constr. 19 (7), 907–916.

Li, J.-q., Chen, X.-l., Duan, P.-y., Mou, J.-h., 2022a. KMOEA: A knowledge-based multi-objective algorithm for distributed hybrid flow shop in a prefabricated system. IEEE Trans. Ind. Inform..

Li, J.-Q., Du, Y., Gao, K.-Z., Duan, P.-Y., Gong, D.-W., Pan, Q.-K., Suganthan, P.N., 2022b. A hybrid iterated greedy algorithm for a crane transportation flexible job shop problem. IEEE Trans. Autom. Sci. Eng. 19 (3), 2153–2170. http://dx.doi.org/10.1109/TASE.2021.3062979.

Li, Y., Gu, W., Yuan, M., Tang, Y., 2022c. Real-time data-driven dynamic scheduling for flexible job shop with insufficient transportation resources using hybrid deep Q network. Robot. Comput.-Integr. Manuf. 74, 102283.

Li, J., Han, Y., Gao, K., Xiao, X., Duan, P., 2023. Bi-population balancing multi-objective algorithm for fuzzy flexible job shop with energy and transportation. IEEE Trans. Autom. Sci. Eng. http://dx.doi.org/10.1109/TASE.2023.3300922.

Lin, C.-C., Deng, D.-J., Chih, Y.-L., Chiu, H.-T., 2019. Smart manufacturing scheduling with edge computing using multiclass deep Q network. IEEE Trans. Ind. Inform. 15 (7), 4276–4284.

Lin, J., Li, Y.-Y., Song, H.-B., 2022. Semiconductor final testing scheduling using Q-learning based hyper-heuristic. Expert Syst. Appl. 187, 115978.

Liu, R., Piplani, R., Toro, C., 2022. Deep reinforcement learning for dynamic scheduling of a flexible job shop. Int. J. Prod. Res. 1–21.

Liu, Z., Zhang, Y., Yu, M., Zhou, X., 2017. Heuristic algorithm for ready-mixed concrete plant scheduling with multiple mixers. Autom. Constr. 84, 1–13.

Luo, S., 2020. Dynamic scheduling for flexible job shop with new job insertions by deep reinforcement learning. Appl. Soft Comput. 91, 106208.

Luo, Q., Deng, Q., Gong, G., Zhang, L., Han, W., Li, K., 2020. An efficient memetic algorithm for distributed flexible job shop scheduling problem with transfers. Expert Syst. Appl. 160, 113721.

Luo, H., Du, B., Huang, G.Q., Chen, H., Li, X., 2013. Hybrid flow shop scheduling considering machine electricity consumption cost. Int. J. Prod. Econom. 146 (2), 423–439.

Luo, S., Zhang, L., Fan, Y., 2021a. Dynamic multi-objective scheduling for flexible job shop by deep reinforcement learning. Comput. Ind. Eng. 159, 107489.

Luo, S., Zhang, L., Fan, Y., 2021b. Real-time scheduling for dynamic partial-no-wait multiobjective flexible job shop by deep reinforcement learning. IEEE Trans. Autom. Sci. Eng..

Ma, Y., Li, J., Cao, Z., Song, W., Guo, H., Gong, Y., Chee, Y.M., 2022. Efficient neural neighborhood search for pickup and delivery problems. In: International Joint Conference on Artificial Intelligence (IJCAI) 2022. arXiv preprint arXiv:2204.11399.

Ma, Z., Yang, Z., Liu, S., Wu, S., 2018. Optimized rescheduling of multiple production lines for flowshop production of reinforced precast concrete components. Autom. Constr. 95, 86–97.

Meng, L., Zhang, C., Ren, Y., Zhang, B., Lv, C., 2020. Mixed-integer linear programming and constraint programming formulations for solving distributed flexible job shop scheduling problem. Comput. Ind. Eng. 142, 106347.

Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A.A., Veness, J., Bellemare, M.G., Graves, A., Riedmiller, M., Fidjeland, A.K., Ostrovski, G., et al., 2015. Human-level control through deep reinforcement learning. Nature 518 (7540), 529–533.

Najafzad, H., Davari-Ardakani, H., Nemati-Lafmejani, R., 2019. Multi-skill project scheduling problem under time-of-use electricity tariffs and shift differential payments. Energy 168, 619–636.

Pan, Q.-K., Gao, L., Wang, L., 2020. An effective cooperative co-evolutionary algorithm for distributed flowshop group scheduling problems. IEEE Trans. Cybern..

Pan, Z., Wang, L., Wang, J., Lu, J., 2021. Deep reinforcement learning based optimization algorithm for permutation flow-shop scheduling. IEEE Trans. Emerg. Top. Comput. Intell..

Park, I.-B., Park, J., 2021. Scalable scheduling of semiconductor packaging facilities using deep reinforcement learning. IEEE Trans. Cybern..

Qi, R., Li, J.-q., Liu, X.-f., 2023. A knowledge-driven multiobjective optimization algorithm for the transportation of assembled prefabricated components with multi-frequency visits. Autom. Constr. 152, 104944.

Rolf, B., Jackson, I., Müller, M., Lang, S., Reggelin, T., Ivanov, D., 2022. A review on reinforcement learning algorithms and applications in supply chain management. Int. J. Prod. Res. 1–29.

Saberi-Aliabad, H., Reisi-Nafchi, M., Moslehi, G., 2020. Energy-efficient scheduling in an unrelated parallel-machine environment under time-of-use electricity tariffs. J. Clean. Prod. 249, 119393.

Sang, Y., Tan, J., 2022. Intelligent factory many-objective distributed flexible job shop collaborative scheduling method. Comput. Ind. Eng. 164, 107884.

Wang, J., Liu, Y., Ren, S., Wang, C., Wang, W., 2021. Evolutionary game based real-time scheduling for energy-efficient distributed and flexible job shop. J. Clean. Prod. 293, 126093.

Wang, J.-j., Wang, L., 2021. A cooperative memetic algorithm with learning-based agent for energy-aware distributed hybrid flow-shop scheduling. IEEE Trans. Evol. Comput..

Wang, S., Zhu, Z., Fang, K., Chu, F., Chu, C., 2018. Scheduling on a two-machine permutation flow shop under time-of-use electricity tariffs. Int. J. Prod. Res. 56 (9), 3173–3187.

Wu, X., Liu, X., Zhao, N., 2019. An improved differential evolution algorithm for solving a distributed assembly flexible job shop scheduling problem. Mem. Comput. 11 (4), 335–355.

Xiong, F., Chu, M., Li, Z., Du, Y., Wang, L., 2021. Just-in-time scheduling for a distributed concrete precast flow shop system. Comput. Oper. Res. 129, 105204.

Xu, W., Hu, Y., Luo, W., Wang, L., Wu, R., 2021. A multi-objective scheduling method for distributed and flexible job shop based on hybrid genetic algorithm and tabu search considering operation outsourcing and carbon emission. Comput. Ind. Eng. 157, 107318.

Xu, H., Li, X., Ruiz, R., Zhu, H., 2019. Group scheduling with nonperiodical maintenance and deteriorating effects. IEEE Trans. Syst. Man Cybern. Syst. 51 (5), 2860–2872.

Yang, Z., Ma, Z., Wu, S., 2016. Optimized flowshop scheduling of multiple production lines for precast production. Autom. Constr. 72, 321–329.

Yuraszeck, F., Mejía, G., Pereira, J., Vilà, M., 2022. A novel constraint programming decomposition approach for the total flow time fixed group shop scheduling problem. Mathematics 10 (3), 329.

Zhao, Z., Liu, S., Zhou, M., Abusorrah, A., 2020. Dual-objective mixed integer linear program and memetic algorithm for an industrial group scheduling problem. IEEE/CAA J. Autom. Sin. 8 (6), 1199–1209.